F# Prelude port map

Status: map only. No module has been ported. This document classifies the F# Prelude (`components/prelude`, 11,642 lines including the writer) before choosing a first vertical slice. It is not a promise that every F# file can be translated 1:1; several files are host runtime, image encoding, or compiler tooling that should remain outside a self-hosted language Prelude.

The target is a self-hosted **language** Prelude, not a line-by-line copy of the F# interpreter. Portable language behavior moves to Scale; host access stays behind the kernel/adapter boundary.

Scope

In scope: `components/prelude/**/*.fs` only. The feature profile is derived from what that code actually uses, not from the whole F# language and not from the .NET ecosystem.

Out of scope for this port: `components/nu-engine` (the external Nu node), `bases/nu-game`, `projects/game`, the writer smoke tools, and the test harness. Nu may later need features the Prelude does not: attributes, SRTP, computation expressions, async/tasks, reflection, quotations, and deeper .NET interop. Those are deferred until a Nu port is a real work item; they must not expand this profile speculatively.

A feature is added here only with a concrete use in the Prelude. "The whole language has it" is not a justification.

Classification

Module	Lines	Class	Port target
`common.fs`	153	Mixed helper	Token/text/tensor helpers are portable; `Store`/`Tensor` access is host. Mapping exercise first.
`program.fs`	107	Runtime	Term boundary; port after code values and function application.
`code.fs`	66	Runtime	Code value encoding/decoding; port with the term runtime.
`stack.fs`	59	Kernel/host	Stack shuffling is the kernel stack; expose as words, do not port the host helper.
`history.fs`	200	Runtime/ diagnostics	Event stream and rollback; optional after the language core.
`quotation-run.fs`	107	Runtime	Quotation evaluation; port after closures/code values.
`lowering.fs`	610	Compiler	Expression-to-lowered compiler; not the interpreter Prelude. Port for AOT later.
`cell-system.fs`	641	Runtime/image	Closure/environment/class encodings; port the data layer, not F# record plumbing.
`closure.fs`	97	Language	`let`, `lambda`, and closure call; first semantic slice with `control.fs`.

Module	Lines	Class	Port target
`scheduler.fs`	1384	Actor runtime	Spawn, supervision, channels, mailboxes; port with host transport capabilities.
`channel.fs`	264	Actor runtime	Channel state and blocking; port with the actor runtime.
`evaluator/scheduler.fs`	603	Actor semantics	Actor, monitor, registry, and mailbox words; port with `scheduler.fs`.
`value-sets.fs`	884	Language	Sets, predicates, constructors; port after patterns and bindings.
`class-system.fs`	731	Language	Classes, slots, instances, precedence; port after cells and dispatch.
`with-fields.fs`	64	Language	Field binding in a child frame; port after closure/class access.
`condition.fs`	225	Language	Conditions, handlers, restarts; port after control/closure.
`runtime-guard.fs`	157	Language	Runtime guard declarations and checks; port after code values.
`channel-event.fs`	569	Language	Event selection, wrappers, guards; port with channels.
`control.fs`	224	Language	`quotation`, `if`, `apply`, `define`, `control`; first semantic slice.
`ownership.fs`	184	Language	Linear ownership and borrow/consume checks; port after core state.
`bookmark.fs`	82	Diagnostics	Bookmark/rollback; port after history.
`environment.fs`	758	Language	Frames, bindings, lexical lookup, unification; port after records/data.
`pattern.fs`	555	Language	Pattern matching and guards; pattern IR already exists.
`set-algebra.fs`	177	Language	Structural set operations; port after value sets.
`dispatch.fs`	1500	Language	Generic functions, methods, class dispatch; port last in the language core.
`word.fs`	260	Language	Word metaobjects and user words; port after environment.
`evaluator.fs`	509	Runtime/language	Dispatcher and main loop; port after code values, patterns, and exceptions.
`kernel-bridge.fs`	28	Kernel/host	Kernel callback adapter; do not port into prose.
`writer-runtime.fs`	95	Compiler	AOT writer runtime; compiler/tooling.
`writer/il-writer.fs`	349	Compiler	IL writer; JIT-only compiler, not the runtime Prelude.

Class meanings:

1. **Language**: portable Scale semantics; the port must preserve behavior.
2. **Runtime**: interpreter machinery that may be rewritten as Scale state or kernel/host services.
3. **Kernel/host**: stays native or adapter-side; prose must call it through a capability, not copy it.
4. **Compiler**: lowering, writer, and AOT; separate from the interpreter Prelude.
5. **Diagnostics**: useful but not on the first language path.

What the Prelude actually requires

The portable language subset is driven by these F# features:

1. functions, currying, partial application, and higher-order calls;
2. `let`, `let rec`, `fun`, `if`, `match`, `try/with`, loops, and mutable locals;
3. records, discriminated unions, options, results, tuples, lists, arrays, and maps/sets;
4. pattern matching over those values, with guards and exhaustive analysis;
5. modules/namespaces, type annotations, generics, and interfaces where the code uses them;
6. exceptions and the F# core library functions (`List`, `Array`, `Map`, `Set`, `Option`, `Result`, `String`);
7. host services: `Store`, `Tensor`, text, scheduler, sockets, and the .NET BCL.

The last item is the boundary. `Store`/`Tensor`/sockets/BCL must be host words or capabilities; they are not reasons to reimplement .NET in prose.

First target

Start with a **mapping slice**, not a full module: `common.fs` is the shared foundation, but it mixes portable helpers with host `Store`/`Tensor` calls. Split it into:

1. portable token/text helpers and tensor equality/arithmetic;
2. host `Store`/`Tensor` access, which stays in the adapter or kernel.

Then take `code.fs` + `closure.fs` + `control.fs` as the first **semantic vertical slice**: quotation values, `let`/`lambda`, and `apply`. That slice proves function values, closures, and code data without dragging in collections or the class system.

After that, port in dependency order: `pattern.fs`, `environment.fs`, `value-sets.fs`, `set-algebra.fs`, `class-system.fs`, `dispatch.fs`, then the actor/channel runtime. `lowering.fs` and the writer stay on the compiler track.

A 1:1 source translation is not the target; a behavior-preserving Scale port is. The F# Prelude remains the reference and the differential oracle while each slice lands.